ACYCLIC VISITOR Is Like a Sparse Matrix

Just as the VISITOR pattern created a matrix of functions, with the visited type on one axis and the function to be performed on the other. ACYCLIC VISITOR creates the a *sparse* matrix. The visitor classes do not have to implement `visit` functions for each visited derivative. For example, if `Ernie` modems cannot be configured for UNIX, then the `UnixModemConfigurator` will not implement the `ErnieVisitor` interface. Thus the ACYCLIC VISITOR pattern allows us to ignore certain combinations of derivatives and functions. This can sometimes be a useful advantage.

Using VISITOR in Report Generators

A very common use of the VISITOR pattern is to walk large data structures and generate reports. This keeps the data-structure objects from having any report-generation code. New reports can be added by adding new VISITORs, rather than by changing the code in the data structures. This means that reports can be placed in separate components and individually deployed only to those customers who need them.

Consider a simple data structure that represents a bill of materials. (See Figure 28-4.) There is an unlimited number of reports that we could generate from this data structure. For example, we could generate a report of the total cost of an assembly, or we could generate a report that listed all the piece parts in an assembly.

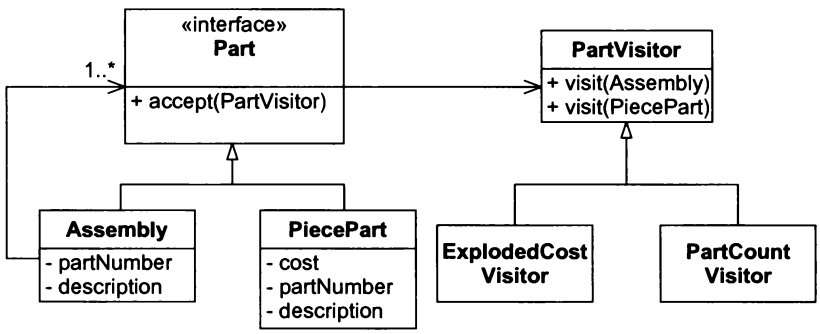


Figure 28-4 Bill-Of-Materials Report-Generator Structure

Each of these reports could be generated by methods in the `Part` class. For example, `getExplodedCost` and `getPieceCount` could be added to the `Part` class. These methods would be implemented in each derivative of `Part`, such that the appropriate reporting was accomplished. Unfortunately, that would mean that every new report that the customers wanted would force us to change the `Part` hierarchy.

The Single Responsibility Principle (SRP) told us that we want to separate code that changes for different reasons. The `Part` hierarchy may change as new kinds of parts are needed. However, it should not change because new kinds of reports are needed. Thus we'd like to separate the reports from the `Part` hierarchy. The VISITOR structure we saw in Figure 28-4 shows how this can be accomplished.

Each new report can be written as a new visitor. We write the `accept` function of `Assembly` to visit the visitor and also call `accept` on all the contained `Part` instances. Thus, the entire tree is traversed. For each node in the tree, the appropriate `visit` function is called on the report. The report accumulates the necessary statistics. The report can then be queried for the interesting data and presented to the user.

This structure allows us to create an unlimited number of reports without affecting the `Part` hierarchy at all. Moreover, each report can be compiled and distributed independently of all the others. This is nice. Listings 28-17 through 28-23 show how this looks in Java.

Listing 28-17

Part.java

```
public interface Part
{
  public String getPartNumber();
  public String getDescription();
  public void accept(PartVisitor v);
}
```

Listing 28-18

Assembly.java

```
import java.util.*;

public class Assembly implements Part
{
  public Assembly(String partNumber, String description)
  {
    itsPartNumber = partNumber;
    itsDescription = description;
  }

  public void accept(PartVisitor v)
  {
    v.visit(this);
    Iterator i = getParts();
    while (i.hasNext())
    {
      Part p = (Part)i.next();
      p.accept(v);
    }
  }
  public void add(Part part)
  {
    itsParts.add(part);
  }

  public Iterator getParts()
  {
    return itsParts.iterator();
  }

  public String getPartNumber()
  {
    return itsPartNumber;
  }

  public String getDescription()
  {
    return itsDescription;
  }

  private List itsParts = new LinkedList();
  private String itsPartNumber;
  private String itsDescription;
```

Listing 28-19

PiecePart.java

```
public class PiecePart implements Part
{
  public PiecePart(String partNumber,
                   String description,
                   double cost)
  {
    itsPartNumber = partNumber;
    itsDescription = description;
    itsCost = cost;
  }

  public void accept(PartVisitor v)
  {
    v.visit(this);
  }

  public String getPartNumber()
  {
    return itsPartNumber;
  }

  public String getDescription()
  {
    return itsDescription;
  }

  public double getCost()
  {
    return itsCost;
  }

  private String itsPartNumber;
  private String itsDescription;
  private double itsCost;
}
```

Listing 28-20

PartVisitor.java

```
public interface PartVisitor
{
  public void visit(PiecePart pp);
  public void visit(Assembly a);
}
```

Listing 28-21

ExplodedCostVisitor.java

```
public class ExplodedCostVisitor implements PartVisitor
{
  private double cost = 0;
  public double cost() {return cost;}
```

```
  public void visit(PiecePart p)
  {cost += p.getCost();}

  public void visit(Assembly a) {}

}
```

Listing 28-22

PartCountVisitor.java

```
import java.util.*;

public class PartCountVisitor implements PartVisitor
{
  public void visit(PiecePart p)
  {
    itsPieceCount++;
    String partNumber = p.getPartNumber();
    int partNumberCount = 0;
    if (itsPieceMap.containsKey(partNumber))
    {
      Integer carrier = (Integer)itsPieceMap.get(partNumber);
      partNumberCount = carrier.intValue();
    }
    partNumberCount++;
    itsPieceMap.put(partNumber, new Integer(partNumberCount));
  }

  public void visit(Assembly a)
  {
  }

  public int getPieceCount() {return itsPieceCount;}
  public int getPartNumberCount() {return itsPieceMap.size();}
  public int getCountForPart(String partNumber)
  {
    int partNumberCount = 0;
    if (itsPieceMap.containsKey(partNumber))
    {
      Integer carrier = (Integer)itsPieceMap.get(partNumber);
      partNumberCount = carrier.intValue();
    }
    return partNumberCount;
  }

  private int itsPieceCount = 0;
  private HashMap itsPieceMap = new HashMap();

}
```

Listing 28-23

TestBOMReport.java

```
import junit.framework.*;
import java.util.*;
```

```
public class TestBOMReport extends TestCase
{
  public TestBOMReport(String name)
  {
    super(name);
  }

  private PiecePart p1;
  private PiecePart p2;
  private Assembly a;

  public void setUp()
  {
    p1 = new PiecePart("997624", "MyPart", 3.20);
    p2 = new PiecePart("7734", "Hell", 666);
    a = new Assembly("5879", "MyAssembly");
  }

  public void testCreatePart()
  {
    assertEquals("997624", p1.getPartNumber());
    assertEquals("MyPart", p1.getDescription());
    assertEquals(3.20, p1.getCost(), .01);
  }

  public void testCreateAssembly()
  {
    assertEquals("5879", a.getPartNumber());
    assertEquals("MyAssembly", a.getDescription());
  }

  public void testAssembly()
  {
    a.add(p1);
    a.add(p2);
    Iterator i = a.getParts();
    PiecePart p = (PiecePart)i.next();
    assertEquals(p, p1);
    p = (PiecePart)i.next();
    assertEquals(p, p2);
    assert(i.hasNext() == false);
  }

  public void testAssemblyOfAssemblies()
  {
    Assembly subAssembly = new Assembly("1324", "SubAssembly");
    subAssembly.add(p1);
    a.add(subAssembly);

    Iterator i = a.getParts();
    assertEquals(subAssembly, i.next());
  }

  private boolean p1Found = false;
  private boolean p2Found = false;
  private boolean aFound = false;
```

```
public void testVisitorCoverage()
{
  a.add(p1);
  a.add(p2);
  a.accept(new PartVisitor(){
    public void visit(PiecePart p)
    {
      if (p == p1)
        p1Found = true;
      else if (p == p2)
        p2Found = true;
    }

    public void visit(Assembly assy)
    {
      if (assy == a)
        aFound = true;
    }
  });
  assert(p1Found);
  assert(p2Found);
  assert(aFound);
}

private Assembly cellphone;

void setUpReportDatabase()
{
  cellphone = new Assembly("CP-7734", "Cell Phone");
  PiecePart display = new PiecePart("DS-1428", "LCD Display", 14.37);
  PiecePart speaker = new PiecePart("SP-92", "Speaker", 3.50);
  PiecePart microphone = new PiecePart("MC-28", "Microphone", 5.30);
  PiecePart cellRadio = new PiecePart("CR-56", "Cell Radio", 30);
  PiecePart frontCover = new PiecePart("FC-77", "Front Cover", 1.4);
  PiecePart backCover = new PiecePart("RC-77", "RearCover", 1.2);
  Assembly keypad = new Assembly("KP-62", "Keypad");
  Assembly button = new Assembly("B52", "Button");
  PiecePart buttonCover = new PiecePart("CV-15", "Cover", .5);
  PiecePart buttonContact = new PiecePart("CN-2", "Contact", 1.2);
  button.add(buttonCover);
  button.add(buttonContact);
  for (int i=0; i<15; i++)
    keypad.add(button);
  cellphone.add(display);
  cellphone.add(speaker);
  cellphone.add(microphone);
```

```
    cellphone.add(cellRadio);
    cellphone.add(frontCover);
    cellphone.add(backCover);
    cellphone.add(keypad);
  }

  public void testExplodedCost()
  {
    setUpReportDatabase();
    ExplodedCostVisitor v = new ExplodedCostVisitor();
    cellphone.accept(v);
    assertEquals(81.27, v.cost(), .001);
  }

  public void testPartCount()
  {
    setUpReportDatabase();
    PartCountVisitor v = new PartCountVisitor();
    cellphone.accept(v);
    assertEquals(36, v.getPieceCount());
    assertEquals(8, v.getPartNumberCount());
    assertEquals("DS-1428", 1, v.getCountForPart("DS-1428"));
    assertEquals("SP-92", 1, v.getCountForPart("SP-92"));
    assertEquals("MC-28", 1, v.getCountForPart("MC-28"));
    assertEquals("CR-56", 1, v.getCountForPart("CR-56"));
    assertEquals("RC-77", 1, v.getCountForPart("RC-77"));
    assertEquals("CV-15", 15, v.getCountForPart("CV-15"));
    assertEquals("CN-2", 15, v.getCountForPart("CN-2"));
    assertEquals("Bob", 0, v.getCountForPart("Bob"));
  }
}
```

Other Uses of VISITOR

In general, the `Visitor` pattern can be used in any application where there is a data structure that needs to be interpreted many different ways. Compilers often create intermediate data structures that represent syntactically correct source code. These data structures are then used to generate compiled code. One could imagine visitors for each different processor or optimization scheme. One could also imagine a visitor that converted the intermediate data structure into a cross-reference listing or even a UML diagram.

Many applications make use of configuration data structures. One could imagine the different subsystems of the application initializing themselves from the configuration data by walking it with their own particular visitors.

In every case where visitors are used, the data structure being used is independent of the uses to which it is being put. New visitors can be created, existing visitors can be changed, and all can be redeployed to installed sites without the recompilation, or redeployment of the existing data structures. This is the power of the VISITOR.

DECORATOR[5]

The visitor gave us a way to add methods to existing hierarchies without changing those hierarchies. Another pattern that accomplishes this is the DECORATOR.

Consider, once again, the `Modem` hierarchy in Figure 28-1. Imagine that we have an application which has many users. Each user, sitting at his computer, can ask the system to call out to another computer using the computer's modem. Some of the users like to hear their modems dial. Others like their modems to be silent.

We could implement this by querying the user preferences at every location in the code where the modem is dialed. If the user wants to hear the modem, we set the speaker volume high. Otherwise, we turn it off.

```
...
Modem m = user.getModem();
if (user.wantsLoudDial())
  m.setVolume(11); // its one more than 10, isn't it?
m.dial(...);
...
```

The spectre of seeing this stretch of code duplicated hundreds of times throughout the application conjures images of 80-hour weeks and heinous debugging sessions. It is something to be avoided.

Another option would be to set a flag in the `modem` object itself and have the `dial` method inspect it and set the volume accordingly.

```
...
public class HayesModem implements Modem
{
  private boolean wantsLoudDial = false;

  public void dial(...)
  {
    if (wantsLoudDial)
    {
      setVolume(11);
    }
    ...
  }
  ...
}
```

This is better, but it must still be duplicated for every derivative of `Modem`. Authors of new derivatives of `Modem` must remember to replicate this code. Depending on programmers memories is pretty risky business.

We could resolve this with the TEMPLATE METHOD[6] pattern by changing `Modem` from an interface to a class, having it hold the `wantsLoudDial` variable, and having it test that variable in the dial function before it calls the `dialForReal` function.

```
...
public abstract class Modem
{
  private boolean wantsLoudDial = false;

  public void dial(...)
```

5. [GOF95].
6. See "Template Method" on page 162.

```
  {
    if (wantsLoudDial)
    {
      setVolume(11);
    }
    dialForReal(...)
  }

  public abstract void dialForReal(...);
}
```

This is better still, but why should `Modem` be affected by the whims of the user in this way? Why should `Modem` know about loud dialing. Must it then be modified every time the user has some other odd request, like logging out before hangup?

Once again the Common-Closure Principle (CCP) comes into play. We want to separate those things that change for different reasons. We can also invoke the Single-Responsibility Principle (SRP) since the need to dial loudly has nothing to do with the intrinsic functions of `Modem` and should therefore not be part of `Modem`.

DECORATOR solves the issue by creating a completely new class named `LoudDialModem`. `LoudDialModem` derives from `Modem` and delegates to a contained instance of `Modem`. It catches the `dial` function and sets the volume high before delegating. Figure 28-5 shows the structure.

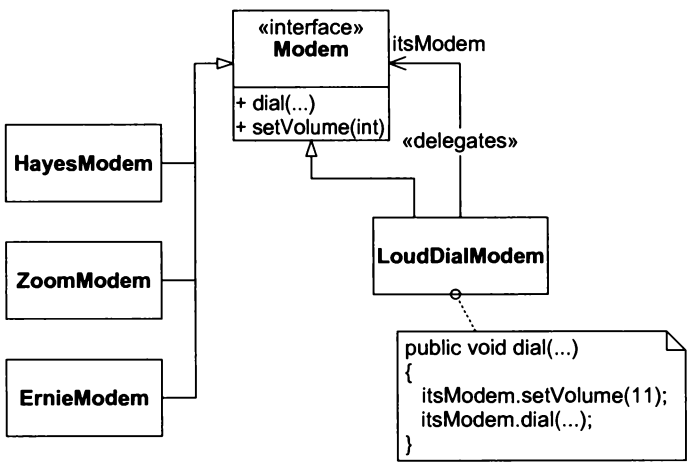


Figure 28-5 DECORATOR: `LoudDialModem`

Now the decision to dial loudly can be made in one place. At the place in the code where the user sets his preferences, if he requests loud dialing, a `LoudDialModem` can be created, and the user's modem can be passed into it. `LoudDialModem` will delegate all calls made to it to the user's modem, so the user won't notice any difference. The `dial` method, however, will first set the volume high before it delegates to the user's modem. The `LoudDialModem` can then become the user's modem without anybody else in the system being affected. Listings 28-24 through 28-27 show the code.

Listing 28-24

Modem.java

```
public interface Modem
{
  public void dial(String pno);
```

```
  public void setSpeakerVolume(int volume);
  public String getPhoneNumber();
  public int getSpeakerVolume();
}
```

Listing 28-25

HayesModem.java

```
public class HayesModem implements Modem
{
  public void dial(String pno)
  {
    itsPhoneNumber = pno;
  }

  public void setSpeakerVolume(int volume)
  {
    itsSpeakerVolume = volume;
  }

  public String getPhoneNumber()
  {
    return itsPhoneNumber;
  }

  public int getSpeakerVolume()
  {
    return itsSpeakerVolume;
  }

  private String itsPhoneNumber;
  private int itsSpeakerVolume;
}
```

Listing 28-26

LoudDialModem.java

```
public class LoudDialModem implements Modem
{
  public LoudDialModem(Modem m)
  {
    itsModem = m;
  }

  public void dial(String pno)
  {
    itsModem.setSpeakerVolume(10);
    itsModem.dial(pno);
  }

  public void setSpeakerVolume(int volume)
  {
    itsModem.setSpeakerVolume(volume);
  }
```

```
  public String getPhoneNumber()
  {
    return itsModem.getPhoneNumber();
  }

  public int getSpeakerVolume()
  {
    return itsModem.getSpeakerVolume();
  }

  private Modem itsModem;
}
```

Listing 28-27

ModemDecoratorTest.java

```
import junit.framework.*;

public class ModemDecoratorTest extends TestCase
{
  public ModemDecoratorTest(String name)
  {
    super(name);
  }

  public void testCreateHayes()
  {
    Modem m = new HayesModem();
    assertEquals(null, m.getPhoneNumber());
    m.dial("5551212");
    assertEquals("5551212", m.getPhoneNumber());
    assertEquals(0, m.getSpeakerVolume());
    m.setSpeakerVolume(10);
    assertEquals(10, m.getSpeakerVolume());
  }

  public void testLoudDialModem()
  {
    Modem m = new HayesModem();
    Modem d = new LoudDialModem(m);
    assertEquals(null, d.getPhoneNumber());
    assertEquals(0, d.getSpeakerVolume());
    d.dial("5551212");
    assertEquals("5551212", d.getPhoneNumber());
    assertEquals(10, d.getSpeakerVolume());

  }
}
```

Multiple Decorators

Sometimes two or more decorators may exist for the same hierarchy. For example, we may wish to decorate the `Modem` hierarchy with `LogoutExitModem`, which sends the string '`exit`' whenever the `Hangup` method is called. This second decorator will have to duplicate all the delegation code that we have already written in `LoudDialModem`. We can eliminate this duplicate code by creating a new class called `ModemDecorator` that

supplies all the delegation code. Then the actual decorators can simply derive from `ModemDecorator` and override only those methods that they need to. Figure 28-6, Listing 28-28, and Listing 28-29 show the structure.

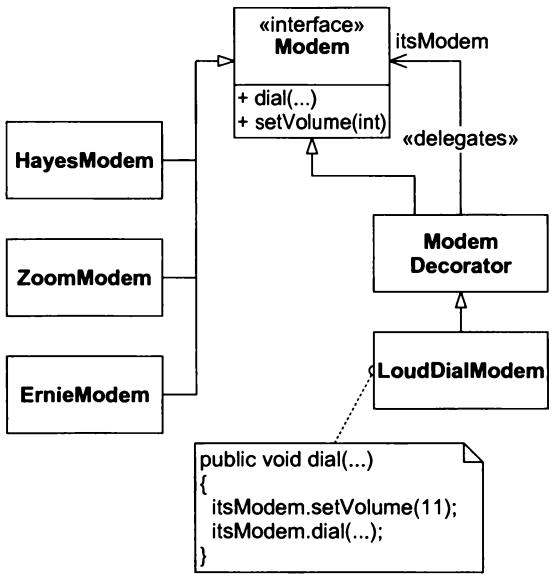


Figure 28-6 `ModemDecorator`

Listing 28-28

`ModemDecorator.java`

```
public class ModemDecorator implements Modem
{
  public ModemDecorator(Modem m)
  {
    itsModem = m;
  }

  public void dial(String pno)
  {
    itsModem.dial(pno);
  }

  public void setSpeakerVolume(int volume)
  {
    itsModem.setSpeakerVolume(volume);
  }

  public String getPhoneNumber()
  {
    return itsModem.getPhoneNumber();
  }

  public int getSpeakerVolume()
  {
    return itsModem.getSpeakerVolume();
  }
```

```
  protected Modem getModem()
  {
    return itsModem;
  }

  private Modem itsModem;
}
```

Listing 28-29

```
public class LoudDialModem extends ModemDecorator
{
  public LoudDialModem(Modem m)
  {
    super(m);
  }

  public void dial(String pno)
  {
    getModem().setSpeakerVolume(10);
    getModem().dial(pno);
  }

}
```

EXTENSION OBJECT

Still another way to add functionality to a hierarchy without changing the hierarchy is to employ the EXTENSION OBJECT[7] pattern. This pattern is more complex than the others, but it is also much more powerful and flexible. Each object in the hierarchy maintains a list of special extension objects. Each object also provides a method that allows the extension object to be looked up by name. The extension object provides methods that manipulate the original hierarchy object.

For example, let's assume that we have a bill-of-materials system again. We need to develop the ability for each object in this hierarchy to create an XML representation of itself. We could put `toXML` methods in the hierarchy, but this would violate the SRP. It may be that we don't want BOM stuff and XML stuff in the same class. We could create XML using a VISITOR, but that doesn't allow us to separate the XML generating code for each type of BOM object. In a VISITOR, all the XML generating code for each BOM class would be in the same VISITOR object. What if we want to separate the XML generation for each different BOM object into its own class?

EXTENSION OBJECT provides a nice way to accomplish this goal. The code in Listings 28-30 through 28-41 shows the BOM hierarchy with two different kinds of extension object. One kind of extension object converts BOM objects into XML. The other kind of extension object converts BOM objects into CSV (comma-separated value) strings. The first kind is accessed by `getExtension("XML")` and the second by `getExtension("CSV")`. The structure is shown in Figure 28-7 and was taken from the completed code. The «marker» stereotype denotes a marker interface (i.e., an interface with no methods).

It is very important to understand that I did not simply write this code in Listings 28-30 through 28-41 from scratch. Rather, I evolved the code from test case to test case. The first source file, Listing 28-30, shows all the test cases. They were written in the order shown. Each test case was written before there was any code that could make it pass. Once each test case was written and failing, the code that made it pass was written. The code was never more complicated than necessary to make the *existing* test cases pass. Thus, the code evolved in tiny increments

7. [PLOPD3], p. 79.

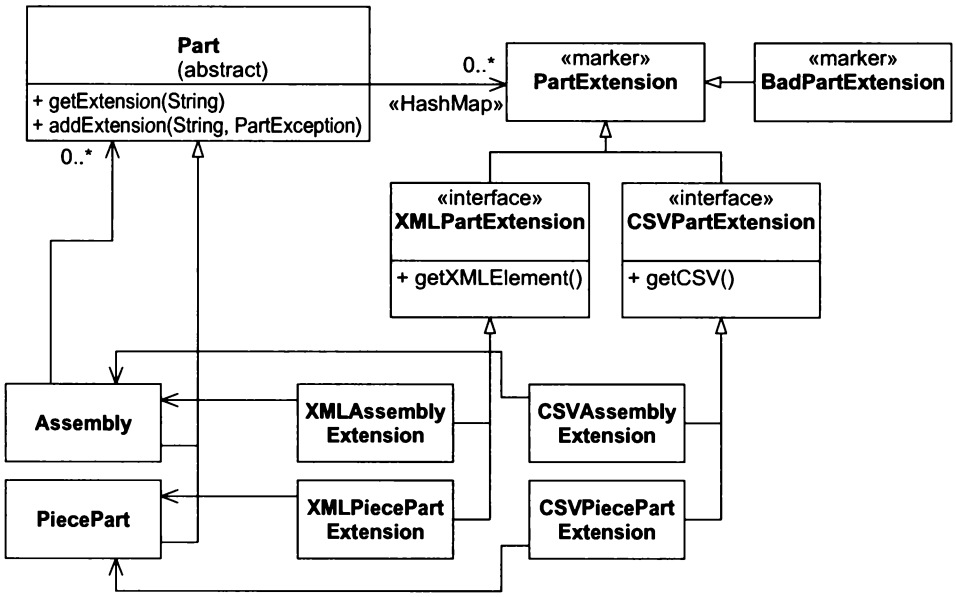


Figure 28-7 Extension Object

from working base to working base. I knew I was trying to build the EXTENSION OBJECT pattern, and I used that to guide the evolution.

Listing 28-30

TestBOMXML.java

```
import junit.framework.*;
import java.util.*;
import org.jdom.*;

public class TestBOMXML extends TestCase
{
  public TestBOMXML(String name)
  {
    super(name);
  }

  private PiecePart p1;
  private PiecePart p2;
  private Assembly a;

  public void setUp()
  {
    p1 = new PiecePart("997624", "MyPart", 3.20);
    p2 = new PiecePart("7734", "Hell", 666);
    a = new Assembly("5879", "MyAssembly");
  }

  public void testCreatePart()
  {
    assertEquals("997624", p1.getPartNumber());
    assertEquals("MyPart", p1.getDescription());
```

```
    assertEquals(3.20, p1.getCost(), .01);
  }

  public void testCreateAssembly()
  {
    assertEquals("5879", a.getPartNumber());
    assertEquals("MyAssembly", a.getDescription());
  }

  public void testAssembly()
  {
    a.add(p1);
    a.add(p2);
    Iterator i = a.getParts();
    PiecePart p = (PiecePart)i.next();
    assertEquals(p, p1);
    p = (PiecePart)i.next();
    assertEquals(p, p2);
    assert(i.hasNext() == false);
  }

  public void testAssemblyOfAssemblies()
  {
    Assembly subAssembly = new Assembly("1324", "SubAssembly");
    subAssembly.add(p1);
    a.add(subAssembly);

    Iterator i = a.getParts();
    assertEquals(subAssembly, i.next());
  }

  public void testPiecePart1XML()
  {
    PartExtension e = p1.getExtension("XML");
    XMLPartExtension xe = (XMLPartExtension)e;
    Element xml = xe.getXMLElement();
    assertEquals("PiecePart", xml.getName());
    assertEquals("997624",xml.getChild("PartNumber").getTextTrim());
    assertEquals("MyPart", xml.getChild("Description").getTextTrim());
    assertEquals(3.2, Double.parseDouble(xml.getChild("Cost").getTextTrim()), .01);
  }

  public void testPiecePart2XML()
  {
    PartExtension e = p2.getExtension("XML");
    XMLPartExtension xe = (XMLPartExtension)e;
    Element xml = xe.getXMLElement();
    assertEquals("PiecePart", xml.getName());
    assertEquals("7734", xml.getChild("PartNumber").getTextTrim());
```

```
    assertEquals("Hell", xml.getChild("Description").getTextTrim());
    assertEquals(666, Double.parseDouble(xml.getChild("Cost").getTextTrim()), .01);
  }

  public void testSimpleAssemblyXML()
  {
    PartExtension e = a.getExtension("XML");
    XMLPartExtension xe = (XMLPartExtension)e;
    Element xml = xe.getXMLElement();
    assertEquals("Assembly", xml.getName());
    assertEquals("5879", xml.getChild("PartNumber").getTextTrim());
    assertEquals("MyAssembly", xml.getChild("Description").getTextTrim());
    Element parts = xml.getChild("Parts");
    List partList = parts.getChildren();
    assertEquals(0, partList.size());
  }

  public void testAssemblyWithPartsXML()
  {
    a.add(p1);
    a.add(p2);
    PartExtension e = a.getExtension("XML");
    XMLPartExtension xe = (XMLPartExtension)e;
    Element xml = xe.getXMLElement();
    assertEquals("Assembly", xml.getName());
    assertEquals("5879", xml.getChild("PartNumber").getTextTrim());
    assertEquals("MyAssembly", xml.getChild("Description").getTextTrim());

    Element parts = xml.getChild("Parts");
    List partList = parts.getChildren();
    assertEquals(2, partList.size());

    Iterator i = partList.iterator();
    Element partElement = (Element)i.next();
    assertEquals("PiecePart", partElement.getName());
    assertEquals("997624", partElement.getChild("PartNumber").getTextTrim());

    partElement = (Element)i.next();
    assertEquals("PiecePart", partElement.getName());
    assertEquals("7734", partElement.getChild("PartNumber").getTextTrim());
  }

  public void testPiecePart1toCSV()
  {
    PartExtension e = p1.getExtension("CSV");
    CSVPartExtension ce = (CSVPartExtension)e;
    String csv = ce.getCSV();
    assertEquals("PiecePart,997624,MyPart,3.2", csv);
  }

  public void testPiecePart2toCSV()
  {
    PartExtension e = p2.getExtension("CSV");
    CSVPartExtension ce = (CSVPartExtension)e;
    String csv = ce.getCSV();
```

```
    assertEquals("PiecePart,7734,Hell,666.0", csv);
  }

  public void testSimpleAssemblyCSV()
  {
    PartExtension e = a.getExtension("CSV");
    CSVPartExtension ce = (CSVPartExtension)e;
    String csv = ce.getCSV();
    assertEquals("Assembly,5879,MyAssembly", csv);
  }

  public void testAssemblyWithPartsCSV()
  {
    a.add(p1);
    a.add(p2);
    PartExtension e = a.getExtension("CSV");
    CSVPartExtension ce = (CSVPartExtension)e;
    String csv = ce.getCSV();

    assertEquals("Assembly,5879,MyAssembly," +
                 "{PiecePart,997624,MyPart,3.2}," +
                 "{PiecePart,7734,Hell,666.0}"
                , csv);
  }

  public void testBadExtension()
  {
    PartExtension pe = p1.getExtension("ThisStringDoesn'tMatchAnyException");
    assert(pe instanceof BadPartExtension);
  }

}
```

Listing 28-31

Part.java

```
import java.util.*;

public abstract class Part
{
  HashMap itsExtensions = new HashMap();

  public abstract String getPartNumber();
  public abstract String getDescription();

  public void addExtension(String extensionType, PartExtension extension)
  {
    itsExtensions.put(extensionType, extension);
  }
```

```
  public  PartExtension getExtension(String extensionType)
  {
    PartExtension pe = (PartExtension) itsExtensions.get(extensionType);
    if (pe == null)
      pe = new BadPartExtension();
    return pe;
  }
}
```

Listing 28-32

PartExtension.java

```
public interface PartExtension
{
}
```

Listing 28-33

PiecePart.java

```
public class PiecePart extends Part
{
  public PiecePart(String partNumber, String description, double cost)
  {
    itsPartNumber = partNumber;
    itsDescription = description;
    itsCost = cost;
    addExtension("CSV", new CSVPiecePartExtension(this));
    addExtension("XML", new XMLPiecePartExtension(this));
  }

  public String getPartNumber()
  {
    return itsPartNumber;
  }

  public String getDescription()
  {
    return itsDescription;
  }

  public double getCost()
  {
    return itsCost;
  }

  private String itsPartNumber;
  private String itsDescription;
  private double itsCost;
}
```

Listing 28-34

Assembly.java

```
import java.util.*;

public class Assembly extends Part
{
  public Assembly(String partNumber, String description)
  {
    itsPartNumber = partNumber;
    itsDescription = description;
    addExtension("CSV", new CSVAssemblyExtension(this));
    addExtension("XML", new XMLAssemblyExtension(this));
  }

  public void add(Part part)
  {
    itsParts.add(part);
  }

  public Iterator getParts()
  {
    return itsParts.iterator();
  }

  public String getPartNumber()
  {
    return itsPartNumber;
  }

  public String getDescription()
  {
    return itsDescription;
  }

  private List itsParts = new LinkedList();
  private String itsPartNumber;
  private String itsDescription;
}
```

Listing 28-35

XMLPartExtension.java

```
import org.jdom.*;

public interface XMLPartExtension extends PartExtension
{
  public Element getXMLElement();
}
```

Listing 28-36

XMLPiecePartException.java

```
import org.jdom.*;

public class XMLPiecePartExtension implements XMLPartExtension
{
  public XMLPiecePartExtension(PiecePart part)
  {
    itsPiecePart = part;
  }

  public Element getXMLElement()
  {
    Element e = new Element("PiecePart");
    e.addContent(
      new Element("PartNumber").setText(
        itsPiecePart.getPartNumber()));
    e.addContent(
      new Element("Description").setText(
        itsPiecePart.getDescription()));
    e.addContent(
      new Element("Cost").setText(
        Double.toString(itsPiecePart.getCost())));
    return e;
  }

  private PiecePart itsPiecePart = null;
}
```

Listing 28-37

XMLAssemblyExtension.java

```
import org.jdom.*;
import java.util.*;

public class XMLAssemblyExtension implements XMLPartExtension
{
  public XMLAssemblyExtension(Assembly assembly)
  {
    itsAssembly = assembly;
  }

  public Element getXMLElement()
  {
    Element e = new Element("Assembly");
    e.addContent(new Element("PartNumber").setText(itsAssembly.getPartNumber()));
    e.addContent(new Element("Description").setText(itsAssembly.getDescription()));
    Element parts = new Element("Parts");
    e.addContent(parts);
    Iterator i = itsAssembly.getParts();
    while (i.hasNext())
    {
      Part p = (Part) i.next();
```

```
      PartExtension pe = p.getExtension("XML");
      XMLPartExtension xpe = (XMLPartExtension)pe;
      parts.addContent(xpe.getXMLElement());
    }
    return e;
  }

  private Assembly itsAssembly = null;
}
```

Listing 28-38

CSVPartExtension.java

```
public interface CSVPartExtension extends PartExtension
{
  public String getCSV();
}
```

Listing 28-39

CSVPiecePartExtension.java

```
public class CSVPiecePartExtension implements CSVPartExtension
{
  private PiecePart itsPiecePart = null;

  public CSVPiecePartExtension(PiecePart part)
  {
    itsPiecePart = part;
  }

  public String getCSV()
  {
    StringBuffer b = new StringBuffer("PiecePart,");
    b.append(itsPiecePart.getPartNumber());
    b.append(",");
    b.append(itsPiecePart.getDescription());
    b.append(",");
    b.append(itsPiecePart.getCost());
    return b.toString();
  }
}
```

Listing 28-40

CSVAssemblyExtension.java

```
import java.util.Iterator;

public class CSVAssemblyExtension implements CSVPartExtension
{
  private Assembly itsAssembly = null;

  public CSVAssemblyExtension(Assembly assy)
  {
    itsAssembly = assy;
  }
```

```
  public String getCSV()
  {
    StringBuffer b = new StringBuffer("Assembly,");
    b.append(itsAssembly.getPartNumber());
    b.append(",");
    b.append(itsAssembly.getDescription());

    Iterator i = itsAssembly.getParts();
    while (i.hasNext())
    {
      Part p = (Part) i.next();
      CSVPartExtension ce = (CSVPartExtension)p.getExtension("CSV");
      b.append(",{");
      b.append(ce.getCSV());
      b.append("}");
    }
    return b.toString();
  }
}
```

Listing 28-41

BadPartExtension.java

```
public class BadPartExtension implements PartExtension
{
}
```

Notice that the extension objects are loaded into each BOM object by that object's constructor. This means that, to some extent, the BOM objects still depend on the XML and CSV classes. If even this tenuous dependency needs to be broken, we could create a FACTORY[8] object that creates the BOM objects and loads their extensions.

The fact that the extension objects can be loaded into the object creates a great deal of flexibility. Certain extension objects can be inserted or deleted from objects depending on the state of the system. It would be very easy to get carried away with this flexibility. For the most part, you probably won't find it necessary. Indeed, the original implementation of `PiecePart.getExtention(String extensionType)` looked like this.

```
public PartExtension getExtension(String extensionType)

{
  if (extensionType.equals("XML"))
    return new XMLPiecePartExtension(this);

  else if (extensionType.equals("CSV"))
    return new XMLAssemblyExtension(this);

  return new BadPartExtension();
}
```

I wasn't particularly thrilled with this because it was virtually identical to the code in `Assembly.getExtension`. The `HashMap` solution in `Part` avoids this duplication and is just simpler. Anyone reading it will know exactly how extension objects are accessed.

8. See "Factory" on page 269.

Conclusion

The VISITOR family of patterns provides us with a number of ways to modify the behavior of a hierarchy of classes without having to change those classes. Thus, they help us maintain the OCP. They also provide mechanisms for segregating different kinds of functionality, thus keeping classes from getting cluttered with many different functions. As such, they help us maintain the CCP. It should be clear that the SRP, LSP, and DIP are also applied to the structure of the VISITOR family.

The VISITOR patterns are seductive. It is easy to get carried away with them. Use them when they help, but maintain a healthy skepticism about their necessity. Often, something that can be solved with a VISITOR can also be solved by something simpler.

Reminder

Now that you've read this chapter, you may wish to go back to Chapter 9, page 107, and solve the problem of ordering the shapes.

Bibliography

1. Gamma, et al. *Design Patterns*. Reading, MA: Addison–Wesley, 1995.
2. Martin, Robert C., et al. *Pattern Languages of Program Design 3*. Reading, MA: Addison–Wesley, 1998.

29

STATE



A state without the means of some change is without the means of its conservation.

—Edmund Burke (1729–1797)

Finite state automata are among the most useful abstractions in the software arsenal. They provide a simple and elegant way to explore and define the behavior of a complex system. They also provide a powerful implementation strategy that is easy to understand and easy to modify. I use them in all levels of a system, from controlling the high-level GUI[1] to the lowest-level communication protocols. They are almost universally applicable.

Overview of Finite State Automata

A simple *finite state machine* (FSM) can be found in the operation of a subway turnstile. This is the device that controls the gate through which passengers pass to gain access to the subway trains. Figure 29-1 shows the beginnings of the FSM that controls the subway turnstile. This diagram is known as a *state transition diagram,* or STD.[2]

1. See "The Taskmaster Architecture" on page 462.
2. See "States and Internal Transitions" on page 493, "Transitions between states" on page 494, and "Nested States" on page 494.

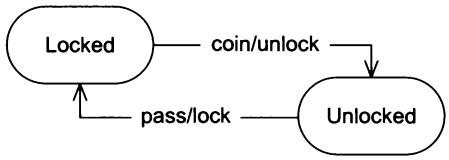


Figure 29-1 Simple Turnstile FSM

STDs are composed of at least four parts. The bubbles are called *states*. Connecting the states are arrows called *transitions*. The transitions are labeled with the name of an *event* followed by the name of an *action*. The STD in Figure 29-1 is read as follows:

- *If* the machine is in the `Locked` state *and* we get a `coin` event, *then* we transition to the `Unlocked` state *and* invoke the `unlock` action.
- *If* the machine is in the `Unlocked` state *and* we get a `pass` event, *then* we transition to the `Locked` state *and* invoke the `lock` action.

These two sentences completely describe the diagram in Figure 29-1. Each sentence describes one transition arrow in terms of four elements: the starting state, the event that triggers the transition, the ending state, and the action to be performed. Indeed these transition sentences can be reduced to a simple table called a *state transition table* (STT). It might look like this:

```
Locked   coin   Unlocked   unlock
Unlocked Pass   Locked     lock
```

How does this machine work? Presume that the FSM begins its life in the `Locked` state. A passenger walks up to the turnstile and deposits a coin. This causes the software to receive the `coin` event. The first transition in the STT says that if we are in the `Locked` state and we get a `coin` event, then we will transition to the `Unlocked` state and invoke the `unlock` action. So the software changes its state to `Unlocked` and calls the `unlock` function. The passenger then passes through the gate causing the software to detect a `pass` event. Since the FSM is now in the `Unlocked` state, the second transition is invoked, forcing the machine back to the `Locked` state and causing the `lock` function to be called.

Clearly the STD and the STT are both simple and elegant descriptions of the behavior of the machine. But they are also very powerful design tools. One of the benefits they convey is the ease with which the designer can detect strange and unhandled conditions. For example, examine each state in Figure 29-1 and apply both known events. Notice that there is no transition to handle a `coin` event in the `Unlocked` state, nor is there a transition to handle a `pass` event in the `Locked` state.

These omissions are serious logic flaws and are a very common source of programmer error. Programmers often consider the normal course of events more thoroughly than they consider the abnormal possibilities. The STD or STT gives the programmer a way to easily check that the design handles every event in every state.

We can fix the FSM by adding the necessary transitions. The new version is shown in Figure 29-2. Here we can see that if the passenger deposits more coins after the first, the machine remains in the `Unlocked` state and lights up a little "thank-you" light encouraging the passenger to continue to enter coins.[3] Also, if the passenger manages to pass through the gate while it is locked (perhaps with the aid of a sledge hammer) then the FSM will remain in the `Locked` state and will sound an alarm.

3. ;^)

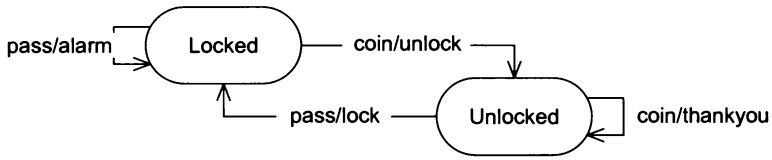


Figure 29-2 Turnstile FSM that covers abnormal events

Implementation Techniques

Nested Switch/Case Statements

There are many different strategies for implementing an FSM. The first, and most direct, is through nested switch/case statements. Listing 29-1 shows one such implementation.

Listing 29-1

Turnstile.java (Nested Switch Case Implementation)

```
package com.objectmentor.PPP.Patterns.State.turnstile;

public class Turnstile
{
  // States
  public static final int LOCKED = 0;
  public static final int UNLOCKED = 1;

  // Events
  public static final int COIN = 0;
  public static final int PASS = 1;

  /*private*/ int state = LOCKED;

  private TurnstileController turnstileController;

  public Turnstile(TurnstileController action)
  {
    turnstileController = action;
  }

  public void event(int event)
  {
    switch (state)
    {
    case LOCKED:
      switch (event)
      {
      case COIN:
        state = UNLOCKED;
        turnstileController.unlock();
        break;
      case PASS:
        turnstileController.alarm();
        break;
      }
      break;
    case UNLOCKED:
```

```
      switch (event)
      {
      case COIN:
        turnstileController.thankyou();
        break;
      case PASS:
        state = LOCKED;
        turnstileController.lock();
        break;
      }
      break;
    }
  }
}
```

The nested switch/case statement divides the code into four mutually exclusive zones, each corresponding to one of the transitions in the STD. Each zone changes the state as needed and then invokes the appropriate action. Thus, the zone for `Locked` and `Coin` changes the state to `Unlocked` and calls `unlock`.

There are some interesting aspects to this code that have nothing to do with the nested switch/case statement. In order for them to make sense, you need to see the unit test that I used to check this code. (See Listings 29-2 and 29-3.)

Listing 29-2

`TurnstileController.java`

```
package com.objectmentor.PPP.Patterns.State.turnstile;

public interface TurnstileController
{
  public void lock();
  public void unlock();
  public void thankyou();
  public void alarm();
}
```

Listing 29-3

`TestTurnstile.java`

```
package com.objectmentor.PPP.Patterns.State.turnstile;

import junit.framework.*;
import junit.swingui.TestRunner;

public class TestTurnstile extends TestCase
{
  public static void main(String[] args)
  {
    TestRunner.main(new String[]{"TestTurnstile"});
  }

  public TestTurnstile(String name)
  {
    super(name);
  }
```

```
  private Turnstile t;
  private boolean lockCalled = false;
  private boolean unlockCalled = false;
  private boolean thankyouCalled = false;
  private boolean alarmCalled = false;

  public void setUp()
  {
    TurnstileController controllerSpoof = new TurnstileController()
    {
      public void lock() {lockCalled = true;}
      public void unlock() {unlockCalled = true;}
      public void thankyou() {thankyouCalled = true;}
      public void alarm() {alarmCalled = true;}
    };

    t = new Turnstile(controllerSpoof);
  }

  public void testInitialConditions()
  {
    assertEquals(Turnstile.LOCKED, t.state);
  }

  public void testCoinInLockedState()
  {
    t.state = Turnstile.LOCKED;
    t.event(Turnstile.COIN);
    assertEquals(Turnstile.UNLOCKED, t.state);
    assert(unlockCalled);
  }

  public void testCoinInUnlockedState()
  {
    t.state = Turnstile.UNLOCKED;
    t.event(Turnstile.COIN);
    assertEquals(Turnstile.UNLOCKED, t.state);
    assert(thankyouCalled);
  }

  public void testPassInLockedState()
  {
    t.state = Turnstile.LOCKED;
    t.event(Turnstile.PASS);
    assertEquals(Turnstile.LOCKED, t.state);
    assert(alarmCalled);
  }

  public void testPassInUnlockedState()
  {
    t.state = Turnstile.UNLOCKED;
    t.event(Turnstile.PASS);
    assertEquals(Turnstile.LOCKED, t.state);
    assert(lockCalled);
  }
}
```

The Package Scope State Variable. Notice the four test functions named `testCoinInLockedState`, `testCoinInUnlockedState`, `testPassInLockedState`, and `testPassInUnlockedState`. These functions test the four transitions of the FSM separately. They do this by forcing the `state` variable of the `Turnstile` to the state they want to check, and then invoking the event they want to verify. In order for the test to access the `state` variable, it cannot be private. Thus, I've made it package scope and added a comment indicating that my intent is that the variable is private.

Object-oriented dogma insists that all instance variables of a class ought to be private. I have blatantly ignored this rule, and by doing so, I have broken the encapsulation of `Turnstile`.

Or have I?

Make no mistake about it, I would rather have kept the `state` variable private. However, to do so would have denied my test code the ability to force its value. I could have created the appropriate `setState` and `getState` methods at package scope, but that seems ridiculous. I was not trying to expose the `state` variable to any class other than `TestTurnstile`, so why should I create a setter and a getter that imply that anyone at package scope can get and set that variable?

One of Java's unfortunate weaknesses is the lack of anything like the C++ `friend` concept. If Java had a `friend` statement, then I could have kept `state` private and declared `TestTurnstile` to be a friend of `Turnstile`. However, as things are, I think that putting `state` at package scope and using the comment to declare my intent is the best option.

Testing the Actions. Notice the `TurnstileController` interface in Listing 29-2. This was put in place specifically so that the `TestTurnstile` class could ensure that the `Turnstile` class was invoking the right action methods in the right order. Without this interface, it would have been much more difficult to ensure that the state machine was working properly.

This is an example of the impact that testing has upon design. Had I simply written the state machine without thought to testing, it is unlikely that I would have created the `TurnstileController` interface. That would have been unfortunate. The `TurnstileController` interface nicely decouples the logic of the finite state machine from the actions it needs to perform. Another FSM, using very different logic, can use the `TurnstileController` without any impact at all.

The need to create test code that verifies each unit in isolation forces us to decouple the code in ways we might not otherwise think of. Thus, testability is a force that drives the design to a less coupled state.

Costs and Benefits of the Nested Switch/Case Implementation. For simple state machines, the nested switch/case implementation is both elegant and efficient. All the states and events are visible on one or two pages of code. However, for larger FSMs the situation changes. In a state machine with dozens of states and events, the code devolves into page after page of case statements. There are no convenient locators to help you see where, in the state machine, you are reading. Maintaining long, nested switch/case statements can be a very difficult and error-prone job.

Another cost of the nested switch/case is that there is no good separation between the logic of the finite state machine and the code that implements the actions. That separation is strongly present in Listing 29-1 because the actions are implemented in a derivative of the `TurnstileController`. However, in most nested switch/case FSMs that I have seen, the implementation of the actions is buried in the case statements. Indeed, this is still possible in Listing 29-1.

Interpreting Transition Tables

A very common technique for implementing FSMs is to create a data table that describes the transitions. This table is interpreted by an engine that handles the events. The engine looks up the transition that matches the event, invokes the appropriate action, and changes the state.

Listing 29-4 shows the code that creates the transition table, and Listing 29-5 shows the transition engine. Both of these listings are snippets from the full implementation in Listing 29-12 at the end of this chapter.

Listing 29-4

Building the turnstile transition table

```
public Turnstile(TurnstileController action)
{
  turnstileController = action;
  addTransition(LOCKED,   COIN, UNLOCKED, unlock()  );
  addTransition(LOCKED,   PASS, LOCKED,   alarm()   );
  addTransition(UNLOCKED, COIN, UNLOCKED, thankyou());
  addTransition(UNLOCKED, PASS, LOCKED,   lock()    );
}
```

Listing 29-5

The transition engine

```
public void event(int event)
  {
    for (int i = 0; i < transitions.size(); i++)
    {
      Transition transition = (Transition) transitions.elementAt(i);
      if (state == transition.currentState && event == transition.event)
      {
        state = transition.newState;
        transition.action.execute();
      }
    }
  }
```

Costs and Benefits of Interpreting a Transition Table. One powerful benefit is that the code that builds the transition table reads like a canonical state transition table. The four `addTransaction` lines can be very easily understood. The logic of the state machine is all in one place and is not contaminated with the implementation of the actions.

Maintaining a finite state machine like this is very easy compared to the nested switch/case implementation. To add a new transition, one simply adds a new `addTransition` line to the `Turnstile` constructor.

Another benefit of this approach is that the table can easily be changed at run time. This allows for dynamic alteration of the logic of the state machine. I have used mechanisms like that to allow hot patching of complex finite state machines.

Still another benefit is that multiple tables can be created, each representing a different FSM logic. These tables can be selected at run time based upon starting conditions.

The cost of the approach is primarily speed. It takes time to search through the transition table. For large state machines, that time may become significant. Another cost is the volume of code that must be written to support the table. If you examine Listing 29-12 closely, you'll see a rather large number of small support functions whose aim is to allow the simple expression of the state transition table in Listing 29-4.